

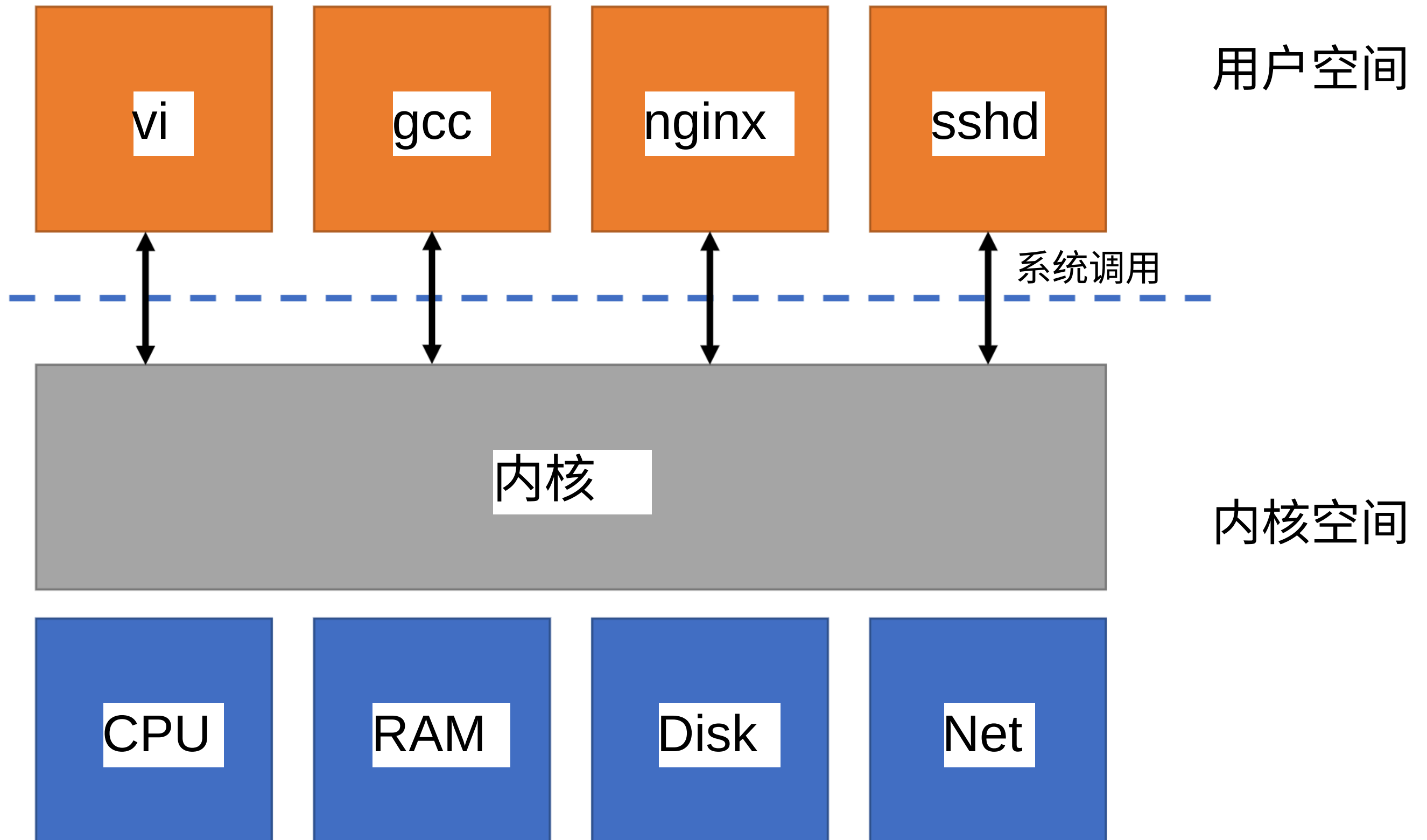
6.S081: OS 组织

Adam Belay <abelay@mit.edu>

讲座主题

- OS 设计
 - 系统调用
 - 硬件隔离
 - 微核内核 vs. 宏核内核
- xv6 中的系统调用

OS 组织（上次的内容）

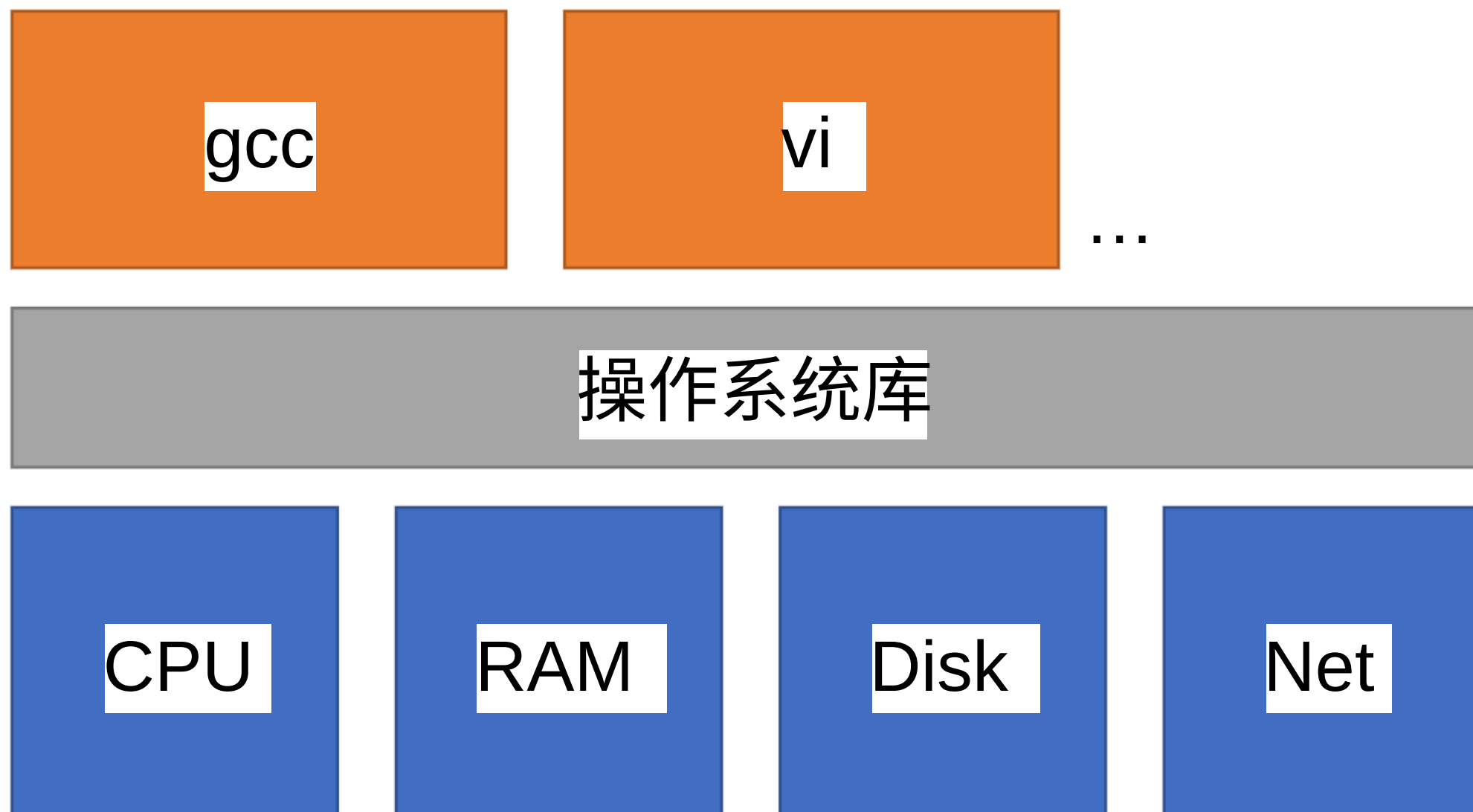


复用

- 必须处理多个应用程序
- 他们之间需要隔离
- 但也必须共享资源

草稿解决方案

- 应用程序直接使用硬件
- 操作系统充当库



问题: 无法复用

- 每个应用必须定期放弃硬件
- 但隔离性较弱
 - 应用忘记放弃 -> 没有其他应用能运行
 - 应用程序有无限循环->其他内容无法运行
 - 即使从另一个应用也无法杀死表现异常的应用
- 该方案有时在实践中被使用
 - 称为协作调度

更大的问题：内存隔离

- 所有应用程序共享物理内存
 - 一个应用程序可以覆盖另一个应用程序的内存
 - 一个应用程序可以覆盖操作系统库
- 没有安全！

UNIX 接口

- 进程（而不是内核）：fork()
 - OS 透明地分配核心
 - 保存 + 恢复寄存器
 - 要求进程放弃它们
 - 周期性地重新分配核心

UNIX 接口

- 内存（而不是物理内存）：exec(), brk()
 - 每个进程都有自己独立的内存
 - 操作系统决定应用程序在内存中的位置
 - 操作系统在应用程序之间实施隔离
 - 操作系统将图像存储在文件系统中（通过 exec 加载）

UNIX 接口

- 文件而不是磁盘块：open(), read(), write()
 - 操作系统提供方便的名称
 - 操作系统允许应用程序和用户共享文件
- 管道而不是共享物理内存：pipe()
 - 操作系统缓冲数据
 - 操作系统如果发送数据过快则停止发送

操作系统必须具有防御性

- 应用程序不应该导致操作系统崩溃
- 应用程序不应该能够突破隔离
- 需要在应用程序和操作系统之间有强大的隔离性

使用 CPU 硬件支持

- 用户/内核模式（特权模式）
- 虚拟内存

CPU 提供用户/内核模式

- 内核模式：可以执行“特权”指令
 - 例如，切换回用户模式
 - 例如，编程定时器芯片
 - 例如，控制虚拟内存
- 用户模式：不能执行“特权”指令
 - 如果尝试执行，会故障到内核模式

计划：内核在内核模式运行，应用程序在用户模式运行

* (本课程不使用 RISC-V M 模式)

CPU 提供虚拟内存

- 页表将虚拟地址转换为物理地址
- 定义应用程序可以访问的物理内存
- 操作系统设置页表，使每个应用程序只能访问其内存

系统调用

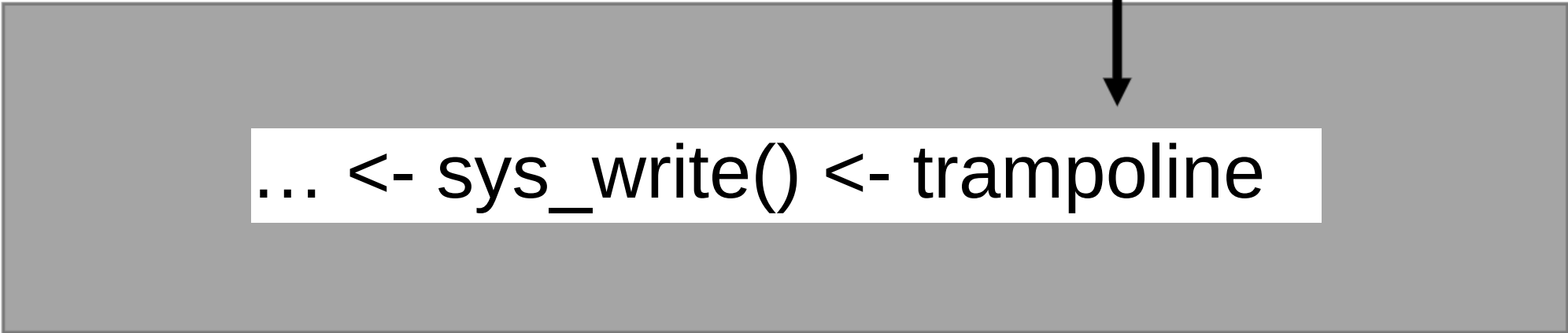
- 应用程序需要与内核通信
- 解决方案是添加指令以受控方式更改模式
 - Risc-V 中的 ecall
 - 在预约定址的指令处进入内核

系统调用

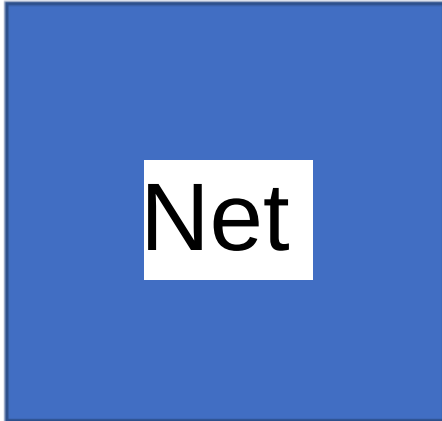
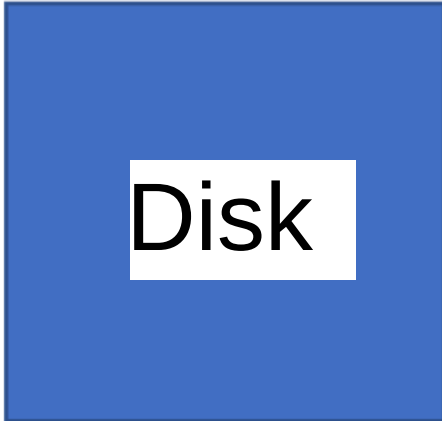
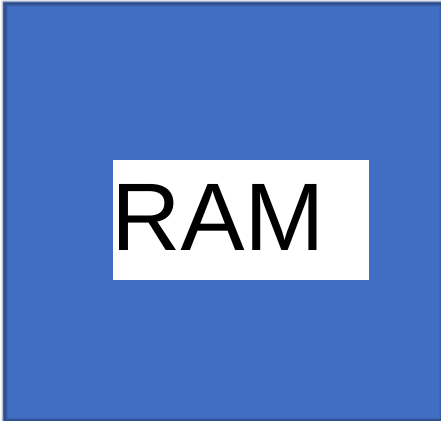
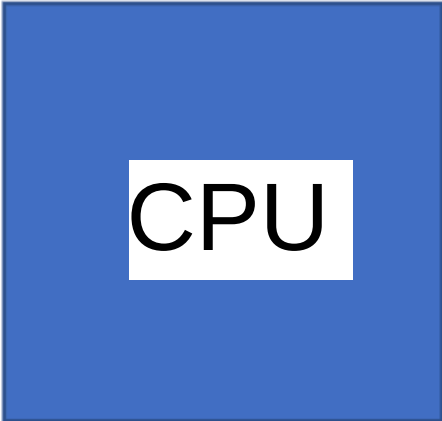


用户空间

系统调用



内核空间



内核是可信赖计算基础

- 核心必须是“正确的”
 - bug 可能会让用户应用绕过隔离
- 核心必须将用户应用视为可疑的
 - 每个系统调用参数必须被验证
 - 用户/内核模式转换必须设置正确
- 需要一种安全思维模式
 - 任何 bug 都可能是一个安全漏洞！

aside: 没有硬件支持的情况下，隔离是否可能？

- 想象一下没有内核/用户模式或虚拟内存
- 好的！使用强类型编程语言
 - 例如，奇点操作系统
- 编译器就是 TCB
- 但最常见的计划是硬件支持

monolithic 内核

- 操作系统运行在内核空间
- xv6 就是这样做的，Linux 也是
- 内核接口 == 系统调用接口
- 核心是一个包含一切的大程序（文件系统、驱动程序、内存管理等）
- 优点：
 - 子系统之间容易协作
 - 例如，一个缓存用于文件系统和虚拟内存
 - 良好的性能
- 弊端：
 - 交互复杂，导致错误
 - 没有隔离性

微内核

- 以普通用户程序运行 OS 服务
 - 例如，服务器提供文件系统
- 内核实现用户空间运行服务的最小机制
 - 具有内存的进程
 - 进程间通信
- 内核接口 != 系统调用接口
- 优点：更多的隔离，更强的容错能力
- 反对：难以获得良好的性能，复杂性

微内核

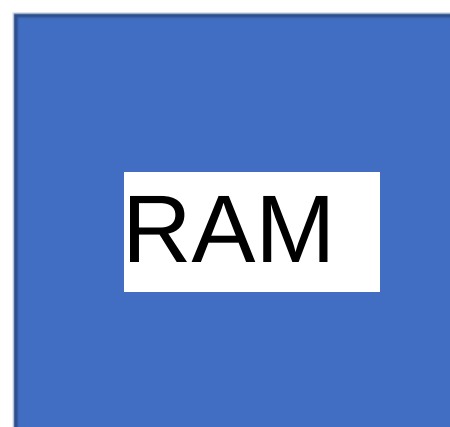
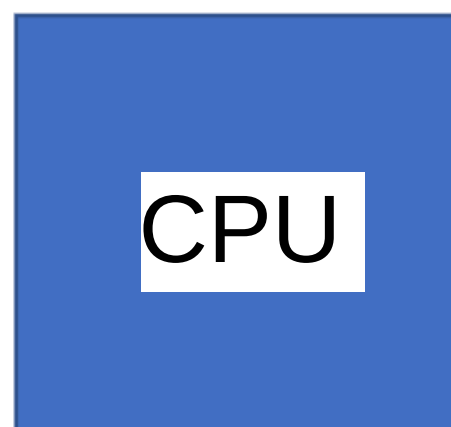


用户空间

系统调用



内核空间

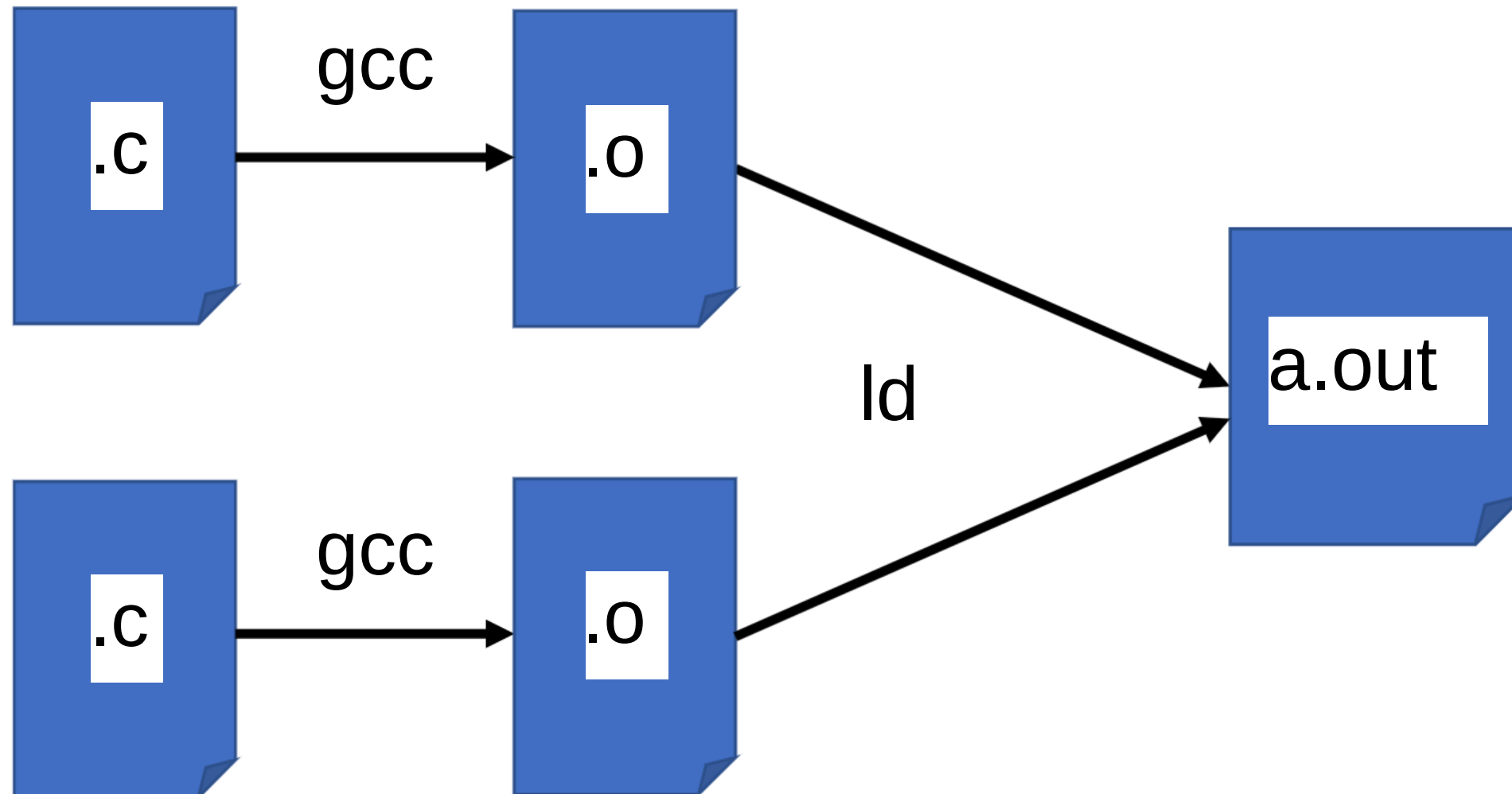


Xv6 案例研究

- 单内核
 - UNIX 系统调用是内核接口
- 源代码反映了操作系统组织结构
 - 用户模式下的应用程序
 - 内核/内核的实现
- 内核有几部分
 - kernel/defs.h, kernel/proc.c, kernel/fs.c, 等。

目标：简单易读/易懂

编译 xv6 (例如, make)



Risc-V（模拟的）计算机

- 一块非常简单的电路板，例如，没有显示
- Risc-V 处理器，具有 N 个核心
- RAM（128 MB）
- 支持中断（PLIC，CLINT）
- 支持 UART（串行端口）
 - Xv6 使用此功能提供控制台（out）
 - Xv6 使用此功能获取键盘输入（输入）
- 支持 E1000 网络卡（通过 PCIe）

为什么使用 qemu 开发？

- 比使用真实硬件更方便
- Qemu 模拟了几种类型的计算机
 - 我们使用 "virt" 版本
<https://github.com/riscv/riscvqemu/wiki>
 - 接近 SiFive 板 (<https://www.sifive.com/boards>) 但使用了 virtio 用于磁盘

CPU 模拟

- 什么是“模拟”？
- qemu 是一个用 C 语言编写的程序，忠实实现了 RISC-V 处理器

```
for (;;) { 读取下一条指令 解码指令 执行指令 （更新  
处理器状态） }
```